SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Department of Computer Science and Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 2 – CO2

CO2 Coverage: Brute-Force and Exhaustive-Search techniques – Selection Sort, Bubble Sort, Insertion Sort, Sequential Search, Brute-Force String Matching, Closest Pair, Convex Hull and Exhaustive Search.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: _____

Q1 – Sorting Under Different Data Conditions 6 Marks

Problem Context – E-Commerce Order Processing

An e-commerce fulfilment centre arranges customer orders according to a **dispatch priority score**.

The development team is considering:

Selection Sort | Bubble Sort | Insertion Sort

However, the incoming data may be randomly ordered, nearly sorted, already sorted or significantly disordered.

Your Challenge

Investigate whether one of the three algorithms should always be preferred or whether the appropriate choice depends on the **characteristics of the input data**.

Before implementation:

- Predict how the three algorithms will behave under different input arrangements.
- Explain the algorithmic reason for your prediction.

Design **two deliberately contrasting datasets** and **one additional edge-case dataset of your own** that will expose meaningful behavioural differences among the algorithms.

Implement the three algorithms and record appropriate measures such as:

- Number of comparisons
- Number of swaps/movements
- Execution time, where useful

Compare your prediction with the observed behaviour.

Design Decision

The fulfilment centre reports that approximately **95% of existing orders are already in the correct priority order**, with only small batches of new orders added periodically.

Determine which of the three algorithms you would recommend and defend your decision.

Evidence Expected

Reasoned prediction, three deliberately designed input conditions, implementation of the three sorting algorithms, operation counts, comparison of predicted and observed behaviour, and an evidence-based recommendation.

Q2 – When Is Sequential Search an Appropriate Engineering Choice? 4 Marks

Problem Context – Aircraft Maintenance Tool Tracking

An aircraft maintenance facility maintains a temporary list of specialised tools currently issued to technicians.

Each record contains:

Tool ID | Technician ID | Issue Time | Work Bay

The list changes continuously as tools are issued and returned.

When a technician asks:

> "Who currently has Tool T-184?"

the system uses **Sequential Search**.

Your Challenge

Design experiments to investigate Sequential Search under:

- Successful searches at different positions
- Unsuccessful searches
- Different list sizes

Record the **number of comparisons** required.

Then answer:

> **Under what operating conditions is Sequential Search a reasonable engineering choice, and when would you stop recommending it?**

Consider both:

- Dataset size

- Frequency of search requests

Changed Condition

The organisation expands from one hangar to **20 maintenance facilities**, resulting in substantially more active records and lookup requests.

Re-evaluate your original recommendation.

Evidence Expected

Designed search experiments, comparison-count evidence, analysis of successful and unsuccessful searches, an evidence-based operating boundary and a justified response to the changed operating condition.

Q3 – Engineer the Behaviour of Brute-Force String Matching 6 Marks

Problem Context – Cybersecurity Log Screening

A security operations centre searches large text-based system logs for known event signatures represented as character patterns.

The initial system uses **Brute-Force String Matching**.

Your Challenge

Instead of merely implementing the algorithm, investigate **how the structure of the text and pattern influences computational effort**.

Design:

1. One text-pattern pair in which the algorithm performs relatively little work.
2. One text-pattern pair deliberately engineered to cause a **large number of character comparisons**.

3. One additional test condition of your own choice.

Before executing each test:

- Predict the expected behaviour.
- Explain the algorithmic reason for your prediction.

Implement the algorithm and record the number of character comparisons.

Inquiry

Compare your prediction with the experimental result.

Explain:

- Why one input requires substantially more work than another
- Which characteristics of the text and pattern create repeated comparisons
- What this reveals about evaluating an algorithm using only its input size

Evidence Expected

Correct implementation, three deliberately designed test conditions, prediction before execution, character-comparison evidence, and a reasoned explanation of the observed differences.

Q4 – Closest Pair and the Cost of Examining Every Possibility 6 Marks

Problem Context – Drone Collision-Risk Monitoring

A logistics company operates delivery drones. At a particular instant, each drone reports its position:

[
D_i=(x_i,y_i)
]

The monitoring system must identify the **two drones closest to one another**.

Your Challenge

Design a **Brute-Force Closest-Pair** solution.

Before implementation, determine:

- Which candidate pairs must be examined
- How duplicate pair comparisons can be avoided
- How many unique pairs exist for (n) drones

Derive the number of candidate pairs mathematically.

Implement your solution and test it using **at least three meaningfully different input sizes**.

Record:

- Number of pairs examined
- Number of distance calculations
- Execution time, where useful

Compare the theoretical growth with experimental behaviour.

Scalability Inquiry

Define what you consider an **acceptable response time** for your experimental environment.

Using your evidence, determine approximately when the brute-force approach begins to become impractical.

Finally, identify **where the computational inefficiency originates** and describe what a redesigned strategy would need to avoid or reduce.

You are not required to implement an advanced Closest-Pair algorithm in this question.

Evidence Expected

Algorithm design, mathematical pair-count derivation, implementation, experimental growth evidence, a defined acceptability criterion and reasoned identification of the limitation of the brute-force strategy.

Q5 – Derive a Convex Hull Algorithm from a Geometric Principle 7 Marks

Problem Context – Wildlife Reserve Boundary Monitoring

Monitoring stations are positioned throughout a wildlife reserve. Each station is represented by coordinates:

[
P_i=(x_i,y_i)
]

Conservation officers need to determine the **outer boundary formed by the monitoring stations**.

You are required to develop a **Brute-Force Convex Hull** solution.

Design Starting Point

You are given only the following geometric principle:

> **A line joining two candidate points may form part of the outer boundary if all remaining points lie on an appropriate single side of that line.**

You are **not provided with the complete algorithm**.

Your Challenge

Convert this geometric property into a computational procedure.

Determine:

- How two candidate points define a line
- How the position of another point relative to that line can be determined
- How the remaining points must be examined

- What condition identifies a potential hull edge
- How the final boundary will be constructed

Develop your own pseudocode before implementation.

Test Your Design

Test the algorithm using:

- A point set containing both boundary and interior points
- A second point configuration deliberately designed by you to challenge your algorithm

Visualise the resulting boundary.

Analyse how the computational work grows as the number of points increases.

Correctness Inquiry

Explain:

> **Why does your geometric test provide evidence that an accepted edge belongs to the outer boundary?**

Your explanation should be based on geometric reasoning rather than simply referring to the program output.

Evidence Expected

A student-derived geometric decision rule, pseudocode, implementation, deliberately designed test configurations, computational analysis, boundary visualisation and a reasoned correctness argument.

Q6 – Exhaustive Search and Combinatorial Growth 8 Marks

Problem Context – Humanitarian Airlift Planning

A helicopter carrying emergency supplies has a strict payload limit.

Each available package contains:

Package ID | Weight | Humanitarian Value

The objective is to select a combination of packages that:

- Does not exceed the payload limit, and
- Maximises the total humanitarian value delivered.

Your Challenge

Design an **Exhaustive-Search strategy**.

Determine:

- How candidate package combinations will be represented
- How candidate solutions will be generated
- How feasibility will be checked
- How feasible solutions will be evaluated
- How the best solution will be retained Do not begin from supplied pseudocode.

Correctness Check

Create a small instance for which the possible combinations can be independently verified.

Use this to establish whether your algorithm identifies the correct optimum.

Growth Investigation

Investigate increasing problem sizes.

Before experimentation:

- Derive/predict how the number of candidate combinations grows with (n). ● Predict what will happen as additional packages are introduced.

Use **at least three meaningful problem sizes** to validate your prediction experimentally.

Inquiry

Explain:

> **Why can adding what appears to be a relatively small number of additional packages produce a dramatic increase in computational effort?**

Your explanation must relate to the growth of the **solution space**.

Deployment Decision

Assume that the airlift planner must produce a loading decision within a fixed response time selected and justified by you.

Using your experimental evidence, estimate the largest problem size for which you would be willing to deploy your exhaustive-search solution.

Evidence Expected

Student-designed exhaustive enumeration, correctness verification, mathematical prediction of solution-space growth, experimental validation, explanation of combinatorial growth and an evidence-based deployment boundary.

DAY-4 DESIGN CHALLENGE

Q7 – Diagnose, Redesign and Validate 13 Marks

Review the evidence you generated while working on Q1–Q6.

Select **ONE solution** that demonstrates a significant computational limitation as its problem size or operating requirement increases.

You are **not permitted simply to replace the solution with a more advanced textbook algorithm and state that its complexity is better**.

Your task is to understand the computational problem first and then develop an appropriate redesign.

A. Diagnose the Computational Bottleneck 2 Marks

Using your own mathematical and/or experimental evidence, identify:

- Where most computational effort is being spent
- Which operation grows significantly as the input increases
- Whether work is being unnecessarily repeated
- Which characteristic of the original algorithm causes the limitation

Evidence Expected

A precise diagnosis supported by evidence from your original implementation and analysis.

B. Define the Design Objective 2 Marks

Before proposing a new solution, clearly state what your redesign must achieve.

For example:

- Reduce unnecessary comparisons

- Avoid examining every candidate
- Exploit existing ordering
- Reuse previously computed information
- Reduce the number of candidate solutions
- Reduce repeated processing

Specify a **measurable objective** wherever possible.

Evidence Expected

A clearly stated redesign objective derived directly from the identified computational limitation.

C. Design the Revised Computational Strategy 4 Marks

Develop a revised solution.

Explain:

- What you have changed
- What information or property your new design exploits
- Which operations are avoided or reduced
- Whether preprocessing is introduced
- Whether additional memory is required
- Any new trade-offs created by the redesign

Provide appropriate pseudocode or a design representation.

Your marks will be based on the **quality of the algorithmic reasoning**, not merely on whether you name a known algorithm.

Evidence Expected

A coherent redesigned computational strategy directly addressing the diagnosed bottleneck, supported by appropriate design representation and technical justification.

D. Predict Before Implementing 1 Mark

Before testing the redesigned solution:

- Predict how its behaviour should differ from the original.
- Identify the performance measure in which you expect improvement.

Evidence Expected

A clear, testable prediction made before experimental validation.

E. Implement and Validate the Redesign 2 Marks

Implement a prototype of the redesigned strategy wherever feasible.

Using the **same or equivalent test conditions**, compare:

Original Solution → Redesigned Solution

Use appropriate measures such as:

- Comparisons
- Key operations
- Candidate solutions examined
- Distance calculations
- Character comparisons
- Execution time, where relevant

Evidence Expected

Comparable experimental evidence showing the behaviour of the original and redesigned solutions.

F. Defend the Final Design 2 Marks

Evaluate whether the redesign achieved its intended objective.

Discuss:

- Whether the prediction was supported
- Magnitude of improvement, if any
- New costs or trade-offs
- Situations in which the original solution may still be preferable
- Whether the redesigned solution sufficiently addresses the identified limitation

A redesign that does **not** outperform the original will not automatically be considered unsuccessful if you correctly analyse and explain why the design hypothesis did not produce the expected result.

Evidence Expected

A critical, evidence-based evaluation of the redesigned solution and a defensible final engineering conclusion.

MARK DISTRIBUTION

Question	Assessed Area	Marks
Q1	Selection, Bubble & Insertion Sort	**6**
Q2	Sequential Search	**4**
Q3	Brute-Force String Matching	**6**
Q4	Brute-Force Closest Pair	**6**
Q5	Brute-Force Convex Hull	**7**

Q6	Exhaustive Search	**8**
Q7	**Day-4 Design Challenge – Diagnose, Redesign & Validate**	**13**
	TOTAL	**50**

Submission Requirements

Submit:

- Your assignment report containing the required evidence for Q1–Q7
- Source code for all implemented solutions
- Raw experimental data used to produce your results
- Tables/graphs used in your analysis, where applicable

Operation counts, comparison counts or other algorithm-specific measures should be treated as **primary experimental evidence**. Execution time may be reported as additional evidence but should not be used as the sole basis for your conclusions.

---

1 – Sorting Under Different Data Conditions

Prediction

Algorithm	Predicted Behaviour
Selection Sort	Performs nearly the same number of comparisons for all input arrangements.
Bubble Sort	Performs efficiently on sorted data when early stopping is used.
Insertion Sort	Performs efficiently on sorted and nearly sorted data.

Test Datasets

- Nearly sorted: [1, 2, 4, 3, 5]
- Reverse sorted: [5, 4, 3, 2, 1]
- Already sorted: [1, 2, 3, 4, 5]

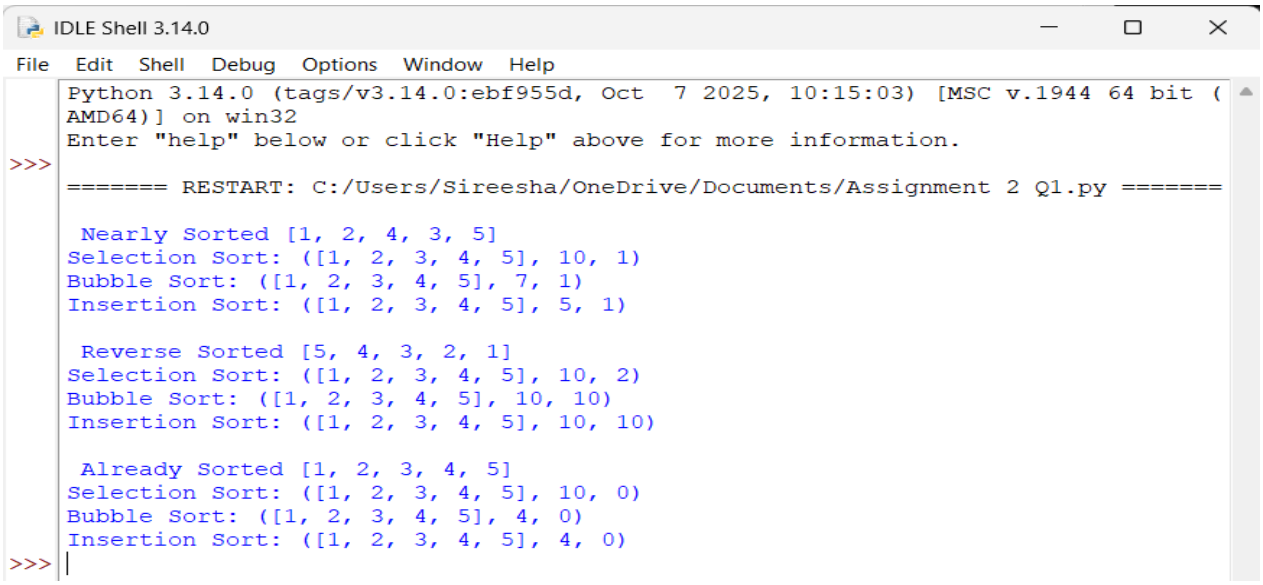
Python Program

```
def selection_sort(arr):
    a = arr.copy()
    comparisons = 0
    swaps = 0
    for i in range(len(a) - 1):
        min_index = i
        for j in range(i + 1, len(a)):
            comparisons += 1
            if a[j] < a[min_index]:
                min_index = j
        if min_index != i:
            a[i], a[min_index] = a[min_index], a[i]
            swaps += 1
    return a, comparisons, swaps
def bubble_sort(arr):
    a = arr.copy()
    comparisons = 0
    swaps = 0
```

```
    n = len(a)
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            comparisons += 1
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swaps += 1
                swapped = True
        if not swapped:
            break
    return a, comparisons, swaps
def insertion_sort(arr):
    a = arr.copy()
    comparisons = 0
    shifts = 0
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0:
            comparisons += 1
            if a[j] > key:
                a[j + 1] = a[j]
                shifts += 1
                j -= 1
            else:
                break
        a[j + 1] = key
    return a, comparisons, shifts
datasets = {
    "Nearly Sorted": [1, 2, 4, 3, 5],
```

```
    "Reverse Sorted": [5, 4, 3, 2, 1],
    "Already Sorted": [1, 2, 3, 4, 5]
}
for name, data in datasets.items():
    print("\nDataset:", name)
    print("Input:", data)
    result = selection_sort(data)
    print("Selection Sort:", result)
    result = bubble_sort(data)
    print("Bubble Sort:", result)
    result = insertion_sort(data)
    print("Insertion Sort:", result)
```

Experimental Results



Dataset	Algorithm	Comparisons	Swaps/Shifts
Nearly sorted	Selection Sort	10	1 swap
Nearly sorted	Bubble Sort	7	1 swap
Nearly sorted	Insertion Sort	5	1 shift
Reverse sorted	Selection Sort	10	2 swaps

Reverse sorted	Bubble Sort	10	10 swaps
Reverse sorted	Insertion Sort	10	10 shifts
Already sorted	Selection Sort	10	0 swaps
Already sorted	Bubble Sort	4	0 swaps
Already sorted	Insertion Sort	4	0 shifts

Observation

Selection Sort performs the same number of comparisons for all datasets. Bubble Sort performs fewer comparisons on sorted data because it stops early. Insertion Sort performs fewer comparisons and shifts when the input is nearly sorted.

Recommendation

I recommend Insertion Sort because approximately 95% of existing orders are already correctly arranged. It efficiently handles nearly sorted data and small batches of new orders. Its best-case time complexity is O(n), making it suitable for this situation.

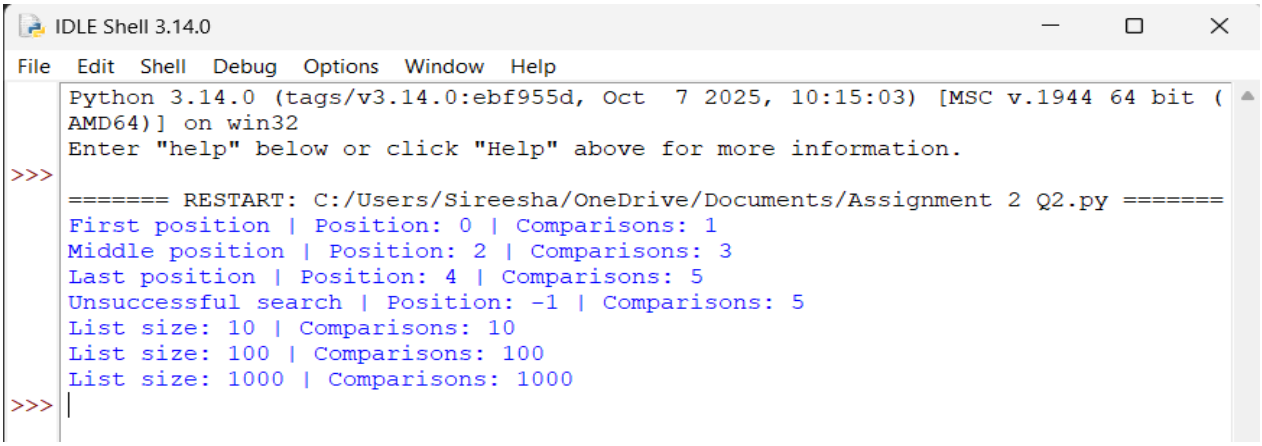
2 – Sequential Search

Python Program

```
def sequential_search(tools, target):
    comparisons = 0
    for i in range(len(tools)):
        comparisons += 1
        if tools[i] == target:
            return i, comparisons
    return -1, comparisons
tools = ["T-101", "T-102", "T-103", "T-104", "T-105"]
tests = [
    ("First position", "T-101"),
    ("Middle position", "T-103"),
    ("Last position", "T-105"),
```

```
    ("Unsuccessful search", "T-999")
]
for name, target in tests:
    position, comparisons = sequential_search(tools, target)
    print(name, "| Position:", position,
          "| Comparisons:", comparisons)
for n in [10, 100, 1000]:
    data = [f"T-{i}" for i in range(n)]
    _, comparisons = sequential_search(data, data[-1])
    print("List size:", n, "| Comparisons:", comparisons)
```

Experimental Results



Search Condition	Comparisons
First position	1
Middle position	3
Last position	5
Unsuccessful search	5

List Size	Comparisons for Last Record
10	10
100	100
1,000	1,000

Recommendation

Sequential Search is suitable for small lists with occasional searches and frequently changing records. Its worst-case time complexity is O(n).

When the organisation expands to 20 facilities, the number of records and search requests increases. A hash table using Tool ID as the key is more suitable because it provides average O(1) lookup time.

3 – Brute-Force String Matching

Test Cases and Predictions

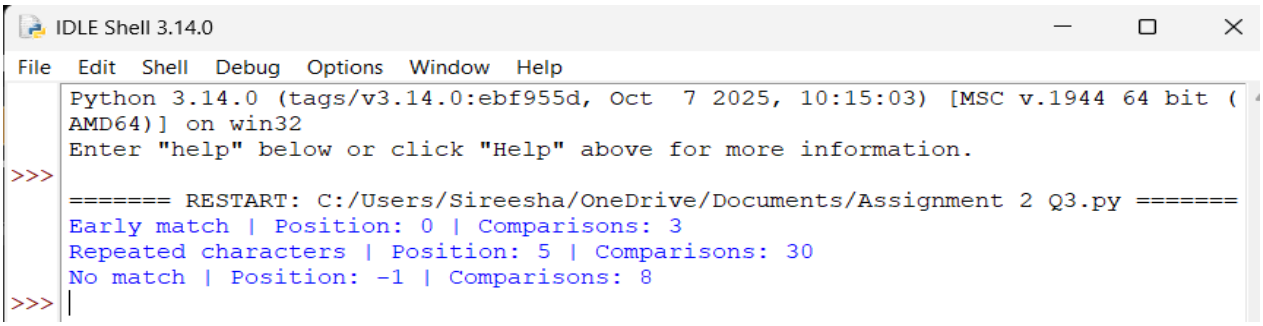
Test	Text	Pattern	Prediction
1	abcdefghi	abc	Few comparisons because the match occurs at the beginning.
2	aaaaaaaaab	aaaab	Many comparisons because of repeated partial matches.
3	abcdefghij	xyz	All possible positions are checked because no match exists.

Python Program

```
def brute_force_matching(text, pattern):
    n = len(text)
    m = len(pattern)
    comparisons = 0
    for i in range(n - m + 1):
        j = 0
        while j < m:
            comparisons += 1
            if text[i + j] != pattern[j]:
                break
            j += 1
        if j == m:
            return i, comparisons
    return -1, comparisons
tests = [
```

```
    ("Early match", "abcdefghi", "abc"),
    ("Repeated characters", "aaaaaaaaab", "aaaab"),
    ("No match", "abcdefghij", "xyz")
]
for name, text, pattern in tests:
    position, comparisons = brute_force_matching(text, pattern)
    print(name, "| Position:", position,
          "| Comparisons:", comparisons)
```

Experimental Results



Test	Result	Comparisons
Early match	Found at index 0	3
Repeated characters	Found at index 5	30
No match	Not found	8

Observation

The early match requires only 3 comparisons. Repeated characters cause more comparisons because the algorithm repeatedly checks matching characters before a mismatch. The unsuccessful search checks all possible starting positions.

The worst-case time complexity is O(nm), where n is the text length and m is the pattern length. Therefore, input structure affects computational effort, not just input size.

4 – Closest Pair and the Cost of Examining Every Possibility

Algorithm

The Brute-Force Closest-Pair algorithm finds the two closest drones by calculating the distance between every unique pair of points. Each pair is examined only once.

The number of unique pairs for n drones is:

$$\text{Pairs} = \frac{n(n-1)}{2}$$

The distance between two points is calculated using:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

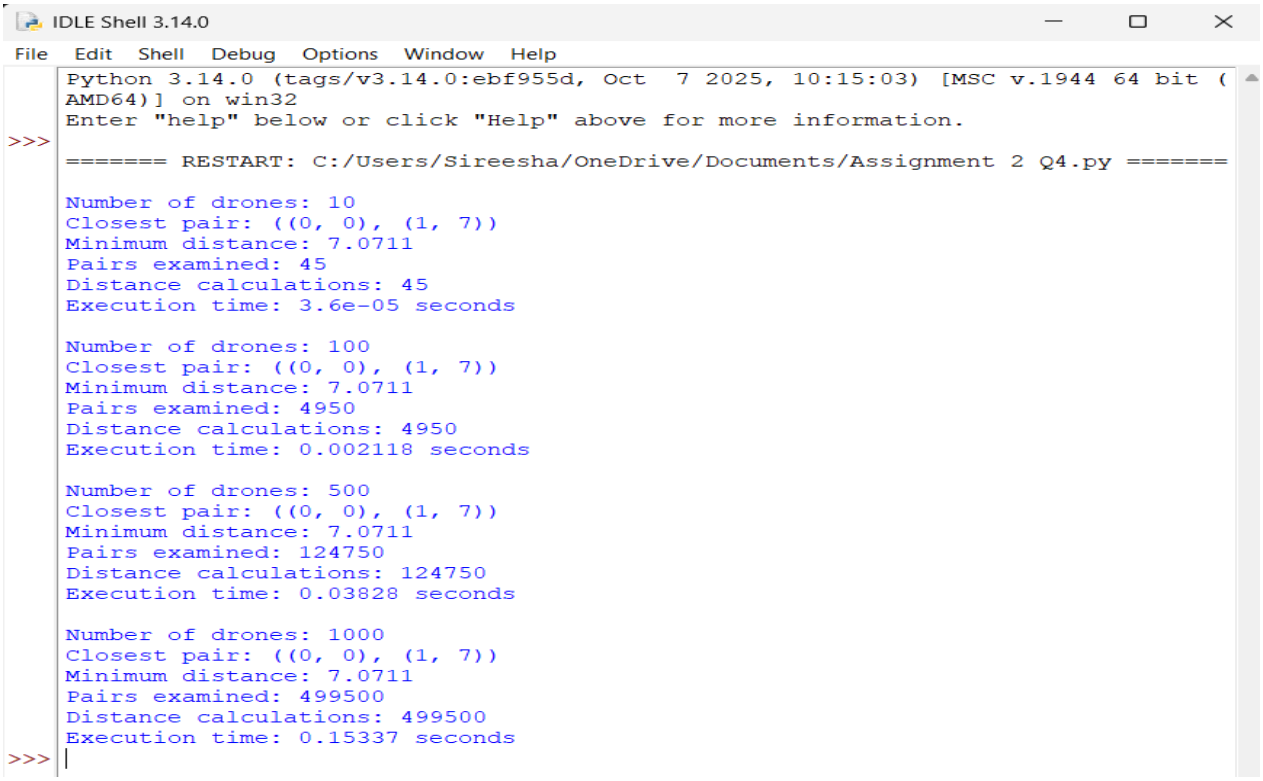
Python Program

```
import math
import time
def closest_pair(points):
    n = len(points)
    min_distance = float("inf")
    closest = None
    pairs_examined = 0
    distance_calculations = 0
    for i in range(n):
        for j in range(i + 1, n):
            pairs_examined += 1
            x1, y1 = points[i]
            x2, y2 = points[j]
            distance = math.sqrt(
                (x2 - x1) ** 2 + (y2 - y1) ** 2
            )
            distance_calculations += 1
            if distance < min_distance:
                min_distance = distance
                closest = (points[i], points[j])
```

```
    return closest, min_distance, pairs_examined, distance_calculations
for n in [10, 100, 500, 1000]:
    points = [(i, (i * 7) % 101) for i in range(n)]
    start = time.perf_counter()
    pair, distance, pairs, calculations = closest_pair(points)
    elapsed = time.perf_counter() - start
    print("\nNumber of drones:", n)
    print("Closest pair:", pair)
    print("Minimum distance:", round(distance, 4))
    print("Pairs examined:", pairs)
    print("Distance calculations:", calculations)
    print("Execution time:", round(elapsed, 6), "seconds")
```

Experimental Results

The number of pairs and distance calculations follows the mathematical formula. Execution times are measured when the program is run.



Number of drones (n)	Pairs examined	Distance calculations
10	45	45
100	4,950	4,950
500	124,750	124,750
1,000	499,500	499,500

Scalability Analysis

The number of pair comparisons grows quadratically, giving the algorithm a time complexity of O(n^2) and auxiliary space complexity of O(1).

For example, 10,000 drones require 49,995,000 distance calculations.

For this experiment, an acceptable response time is defined as 1 second. The measured execution times should be compared with this limit to identify when the algorithm becomes impractical on the computer being used.

Observation and Improvement

The number of distance calculations increases rapidly as the number of drones increases. The main inefficiency is that every possible pair is examined, including pairs that may be far apart.

A redesigned strategy should reduce the number of candidate pairs by using the spatial arrangement of the points to avoid unnecessary distance calculations.

5 – Derive a Convex Hull Algorithm from a Geometric Principle

Algorithm and Geometric Principle

A Brute-Force Convex Hull algorithm identifies the outer boundary of a set of monitoring stations represented by coordinates (x,y).

For every pair of points, consider the line joining them. If all the remaining points lie on the same side of that line, the line may form an edge of the convex hull.

The side of a point relative to a line can be determined using:

$$C = (x_2 - x_1)(y - y_1) - (y_2 - y_1)(x - x_1)$$

- If C>0, the point lies on one side.
- If C<0, the point lies on the opposite side.
- If C=0, the point lies on the line.

Pseudocode

```
CONVEX-HULL(points):
    hull_edges = empty list
    FOR each pair of points A, B:
        positive = false
        negative = false
        FOR each point P other than A and B:
            calculate cross product C
            IF C > 0:
                positive = true
            ELSE IF C < 0:
                negative = true
        IF positive and negative are not both true:
            add edge AB to hull_edges
    construct the outer boundary from hull_edges
    return the hull boundary
```

Python Implementation and Visualization

```
import matplotlib.pyplot as plt
def convex_hull(points):
    edges = []
    n = len(points)
    for i in range(n):
        for j in range(i + 1, n):
            x1, y1 = points[i]
            x2, y2 = points[j]
            positive = False
            negative = False
            for k in range(n):
                if k == i or k == j:
                    continue
```

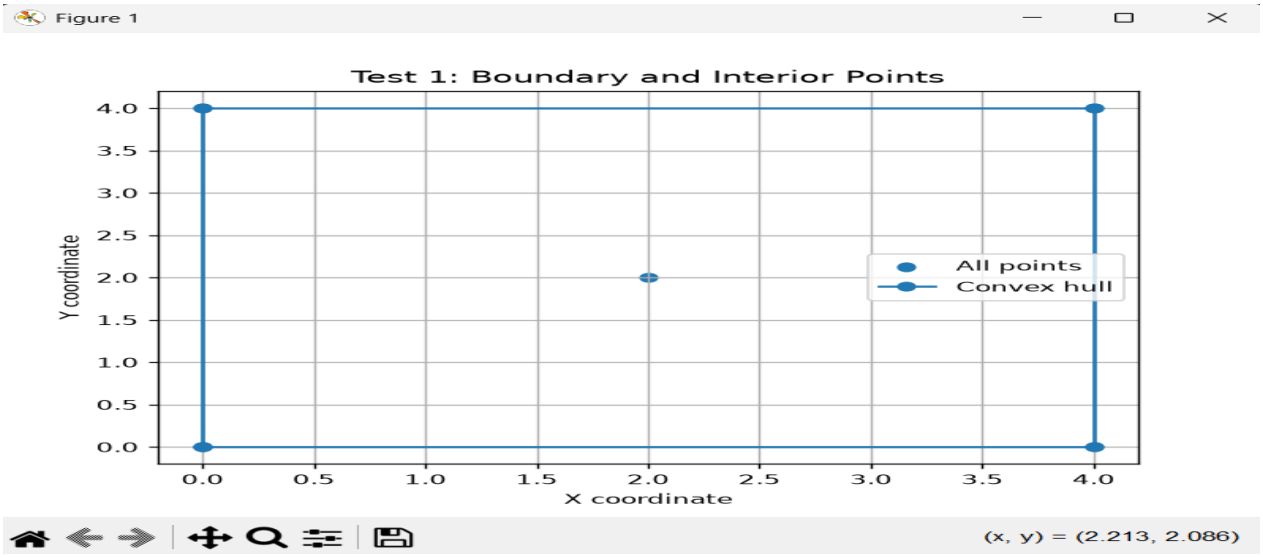
```
            x, y = points[k]
            cross = ((x2 - x1) * (y - y1)
                  - (y2 - y1) * (x - x1))
            if cross > 0:
                positive = True
            elif cross < 0:
                negative = True
        if not (positive and negative):
            edges.append((points[i], points[j]))
    hull_points = list(dict.fromkeys(
        point for edge in edges for point in edge
    ))
    cx = sum(p[0] for p in hull_points) / len(hull_points)
    cy = sum(p[1] for p in hull_points) / len(hull_points)
    hull_points.sort(
        key=lambda p: __import__("math").atan2(
            p[1] - cy, p[0] - cx
        )
    )
    return hull_points, len(edges)
def test_hull(points, title):
    hull, edge_count = convex_hull(points)
    print("\n", title)
    print("Hull points:", hull)
    print("Candidate pairs checked:",
          len(points) * (len(points) - 1) // 2)
    print("Accepted boundary pairs:", edge_count)
    closed = hull + [hull[0]]
    plt.figure()
    plt.scatter(
        [p[0] for p in points],
```

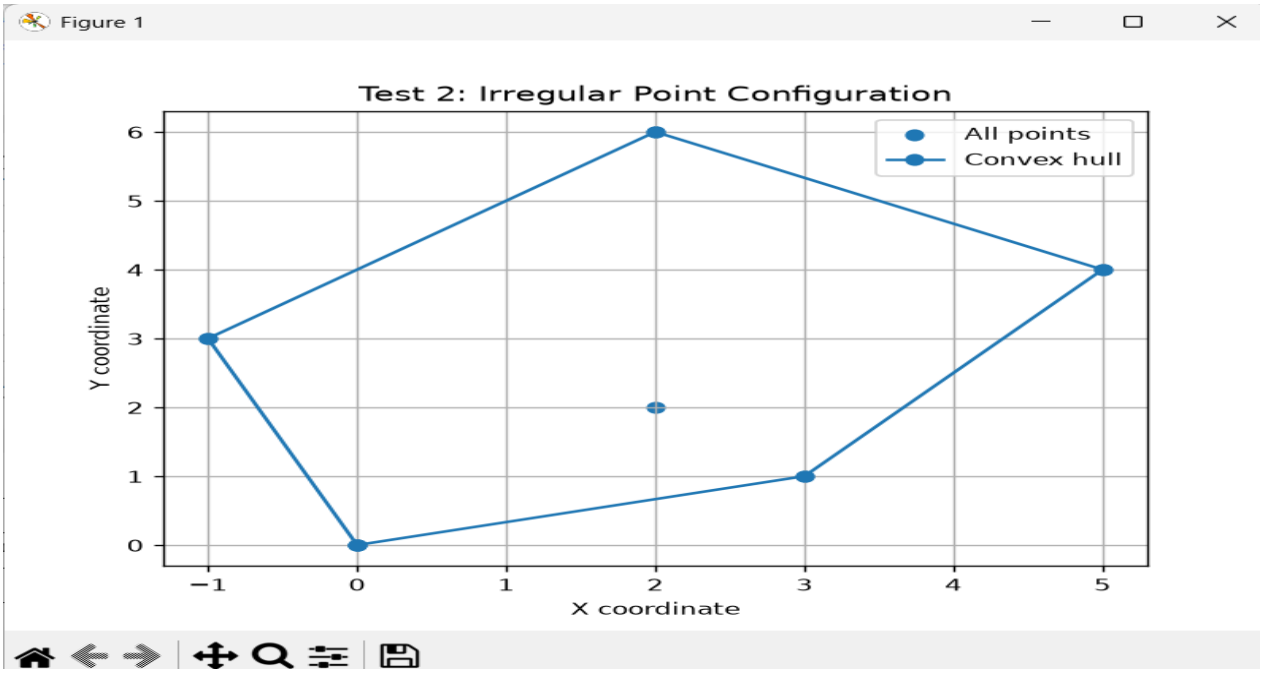
```
        [p[1] for p in points],
        label="All points"
    )
    plt.plot(
        [p[0] for p in closed],
        [p[1] for p in closed],
        marker="o",
        label="Convex hull"
    )
    plt.title(title)
    plt.xlabel("X coordinate")
    plt.ylabel("Y coordinate")
    plt.legend()
    plt.grid(True)
    plt.show()
test_hull(
    [(0, 0), (4, 0), (4, 4), (0, 4), (2, 2)],
    "Test 1: Boundary and Interior Points"
)
test_hull(
    [(0, 0), (3, 1), (5, 4), (2, 6), (-1, 3), (2, 2)],
    "Test 2: Irregular Point Configuration"
)
```

Test Results

```
*IDLE Shell 3.14.0*
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
======= RESTART: C:/Users/Sireesha/OneDrive/Documents/Assignment 2 Q5.py =======

 Test 1: Boundary and Interior Points
Hull points: [(0, 0), (4, 0), (4, 4), (0, 4)]
Candidate pairs checked: 10
Accepted boundary pairs: 4
```





Test case	Number of points	Expected boundary
Boundary and interior points	5	Four corners of the square
Irregular configuration	6	Five outer boundary points

Complexity Analysis

For every pair of points, the algorithm checks the remaining points to determine which side of the line they occupy.

- Candidate pairs: $\frac{n(n-1)}{2}$
- Time complexity: $O(n^3)$
- Auxiliary space complexity: O(n), excluding the visualization.

As the number of points increases, the number of side tests increases rapidly.

Correctness

If all points lie on one side of the line joining two candidate points, that line supports the point set and can form part of the convex hull boundary. Checking every candidate pair identifies the outer boundary edges for the test configurations. Sorting the boundary vertices around their centre displays the hull in order.

6 – Exhaustive Search and Combinatorial Growth

Algorithm

The objective is to select packages that maximize total humanitarian value without exceeding the helicopter's payload limit.

Each package is represented by its ID, weight, and humanitarian value. Exhaustive Search generates every possible package combination, calculates its total weight and value, rejects combinations exceeding the payload limit, and retains the feasible combination with the highest value.

For n packages, the number of possible subsets is:

$$\boxed{2^n}$$

The empty subset is included in this count.

Python Implementation

```
 import time
def exhaustive_search(packages, capacity):
    n = len(packages)
    best_value = 0
    best_combination = []
    combinations_checked = 0
    for mask in range(1 << n):
        total_weight = 0
        total_value = 0
        selected = []
        for i in range(n):
            if mask & (1 << i):
                package_id, weight, value = packages[i]
                total_weight += weight
                total_value += value
                selected.append(package_id)
        combinations_checked += 1
        if total_weight <= capacity and total_value > best_value:
            best_value = total_value
            best_combination = selected
    return best_combination, best_value, combinations_checked
packages = [
    ("P1", 2, 3),
    ("P2", 3, 4),
    ("P3", 4, 5),
    ("P4", 5, 8)
]
capacity = 8
selected, value, checked = exhaustive_search(packages, capacity)
print("Selected packages:", selected)
```

```
print("Maximum humanitarian value:", value)
print("Combinations checked:", checked)
print("\nGrowth Experiment")
for n in [10, 15, 20]:
    test_packages = [
        (f"P{i}", 1, i + 1) for i in range(n)
    ]
    start = time.perf_counter()
    _, _, checked = exhaustive_search(test_packages, n // 2)
    elapsed = time.perf_counter() - start
    print("Packages:", n)
    print("Combinations:", checked)
    print("Execution time:", round(elapsed, 6), "seconds")
```

Correctness Verification

For the test instance, the payload limit is 8 units.

Package	Weight	Humanitarian Value
P1	2	3
P2	3	4
P3	4	5
P4	5	8

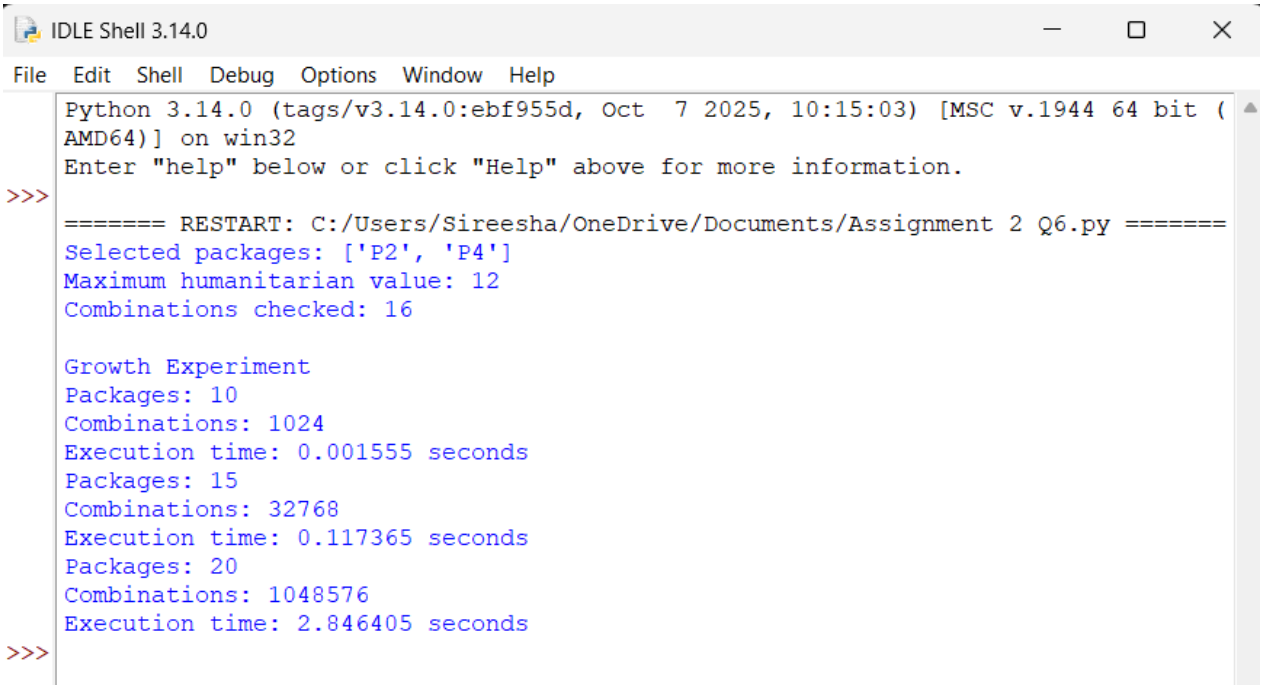
Optimal combination: P2 and P4.

- Total weight = 3+5=8
- Total humanitarian value = 4+8=12
- Total combinations examined = 2^4=16

The combination is feasible and gives the maximum value of 12.

Growth Investigation

Number of packages (n)	Combinations (2^n)
10	1,024
15	32,768
20	1,048,576



Observation and Deployment Decision

The number of combinations doubles whenever one additional package is introduced. Therefore, even a small increase in the number of packages can cause a significant increase in computational effort.

The time complexity is $O(n2^n)$, because every subset may require checking up to n packages. The auxiliary space complexity is O(n) for storing the selected combination.

7 – Diagnose, Redesign and Validate

A. Diagnose the Computational Bottleneck

The Brute-Force Closest-Pair algorithm calculates the distance between every unique pair of drones.

The number of pairs is:

$$\frac{n(n-1)}{2}$$

Number of drones	Distance calculations
10	45
100	4,950
500	124,750
1,000	499,500

The main bottleneck is the large number of distance calculations. The algorithm repeatedly examines pairs that may be far apart and cannot improve the current minimum distance. Its time complexity is $O(n^2)$.

B. Define the Design Objective

The redesigned algorithm should reduce unnecessary distance calculations while still finding the correct closest pair.

The objectives are:

- Avoid comparing points that are too far apart in the x-coordinate.
- Avoid calculating distances when the y-coordinate difference cannot improve the current minimum.
- Maintain the correctness of the closest-pair result.
- Reduce the number of distance calculations compared with the brute-force method.

C. Design the Revised Computational Strategy

Revised strategy: Sort points by x-coordinate and filter unnecessary comparisons.

First, sort the drone positions by their x-coordinate. Maintain the current minimum distance and consider only earlier points whose x-coordinate difference is smaller than that distance. Also skip candidates whose y-coordinate difference is at least the current minimum distance.

This uses the geometric property that if either coordinate difference is greater than or equal to the current minimum distance, the pair cannot produce a smaller Euclidean distance.

Pseudocode

```
REVISED-CLOSEST-PAIR(points):
    sort points by x-coordinate
    minimum_distance = infinity
    closest_pair = none
    distance_calculations = 0

    FOR each point P in sorted points:
        remove earlier points Q where
            P.x - Q.x >= minimum_distance

        FOR each remaining point Q:
            IF absolute(P.y - Q.y) < minimum_distance:
                calculate distance between P and Q
                distance_calculations += 1

                IF distance < minimum_distance:
                    minimum_distance = distance
                    closest_pair = (P, Q)

    RETURN closest_pair, minimum_distance,
           distance_calculations
```

Changes and trade-offs:

- Sorting introduces preprocessing with O(n log n) time.
- Coordinate filtering avoids some unnecessary distance calculations.
- Additional memory is needed to maintain the active points.
- The method remains O(n^2) in the worst case, but can perform better on suitable point distributions.

D. Predict Before Implementing

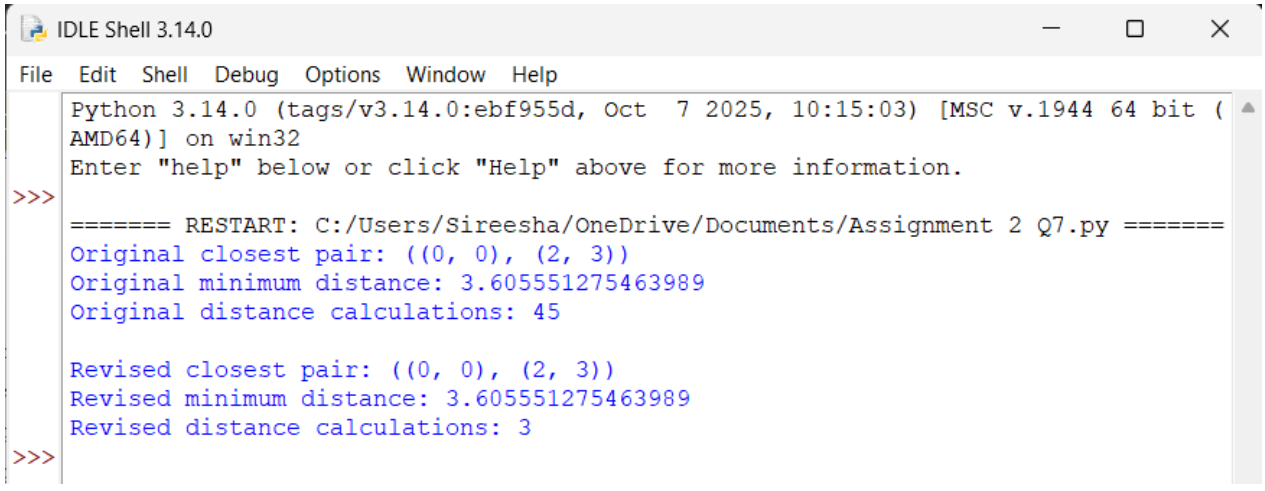
The revised strategy is expected to perform fewer distance calculations when many points can be excluded using coordinate differences. The main performance measure is the number of distance calculations compared with the original brute-force algorithm.

E. Implement and Validate the Redesign

```
import math
def brute_force(points):
    best = float("inf")
    pair = None
    calculations = 0
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            calculations += 1
            d = math.dist(points[i], points[j])
            if d < best:
                best = d
                pair = (points[i], points[j])
    return pair, best, calculations
def revised_closest_pair(points):
    points = sorted(points, key=lambda p: p[0])
    active = []
    best = float("inf")
    pair = None
    calculations = 0
    for p in points:
        active = [
            q for q in active
            if p[0] - q[0] < best
        ]
        for q in active:
            if abs(p[1] - q[1]) < best:
```

```
            calculations += 1
            d = math.dist(p, q)
            if d < best:
                best = d
                pair = (q, p)
        active.append(p)
    return pair, best, calculations
points = [
    (0, 0), (1, 8), (2, 3), (4, 10),
    (7, 4), (8, 12), (12, 1), (15, 9),
    (20, 5), (25, 15)
]
original = brute_force(points)
revised = revised_closest_pair(points)
print("Original closest pair:", original[0])
print("Original minimum distance:", original[1])
print("Original distance calculations:", original[2])
print("\nRevised closest pair:", revised[0])
print("Revised minimum distance:", revised[1])
print("Revised distance calculations:", revised[2])
```

Output



Validation

Run the program and compare the two results. Both methods should return the same minimum distance. The distance-calculation counts show whether coordinate filtering reduces the computational effort for this dataset.

The original method performs $n(n-1)/2$ distance calculations. The revised method performs calculations only for candidates that pass its coordinate filters.

F. Defend the Final Design

The redesigned method is expected to reduce distance calculations when coordinate filtering eliminates many candidate pairs. Its correctness is maintained because any pair with an x- or y-coordinate difference greater than or equal to the current minimum distance cannot improve that distance.

The trade-offs are the additional sorting time and memory required for the active points. In the worst case, the redesigned method can still perform $O(n^2)$ work.

The original brute-force algorithm remains preferable for very small datasets because it is simpler and requires no sorting. The redesigned method is more useful when the dataset is larger and coordinate filtering eliminates many unnecessary comparisons.